

SOFTWARE ENGINEERING

UNIT-I

(Software Product and Software Process)

Dr. Randhir Kumar

COURSE STRUCTURE

- Introduction to software, SE
- Software Development Life Cycles
- Analysis
- Design
- Coding and Testing
- Maintenance
- Software Quality Assurance

EVOLUTION OF SOFTWARE

- 1945-65 (Origin – Build and Fix model)
 - 1965-85 (Software Crisis) – only 2 % software was used (IBM – OS/360)
 - 1985-2000 (Test based development)
 - 2000-2010 – (Application software Development)
 - 2010- Till Date – (AI, Deep Learning, Blockchain, etc.)
-
- First Software was developed on 21 June 1948 by Manchester university, UK . This software was used for finding highest integer power (ex: 2^{18})

WHAT IS SOFTWARE ?

- A program is set of instructions.
- Software can define as:
 - Set of programs – executed provide desire features, function & performance.
 - Documentation Manual
 - Operation Manuals

- Documentation manuals:
 - Requirement Analysis
 - Formal specification, context-diagram, DFD
 - Software Design
 - Flow charts, ER, Structure Chart, UML Diagrams
 - Implementation
 - Source code, Cross Reference listing
 - Testing
 - Test data, Test results
- Operation Manuals :
 - User manuals
 - System overview, Beginners user tutorial, reference guide
 - Operation manuals
 - Installation guide, System administration guide

SOFTWARE MYTHS

- Software is easy to change.
- Testing software can remove all the errors.
- Reusing the software increases the safety.
- Software can work right the first time.
- Software with more features is a better software.
- Addition of more software engineers will make up the delay.
- The available standards and practices are sufficient to develop any type of software

SOFTWARE CHARACTERISTICS

- Software is developed or engineered; it is not manufactured.
- Software does not “wear out” but it does deteriorate.
- Reusability of components.
- Software is flexible.
- Software continues to be custom built.

HARDWARE VS. SOFTWARE

Hardware

- ☐ Manufactured
- ☐ wear out
- ☐ Built using components
- ☐ Relatively simple

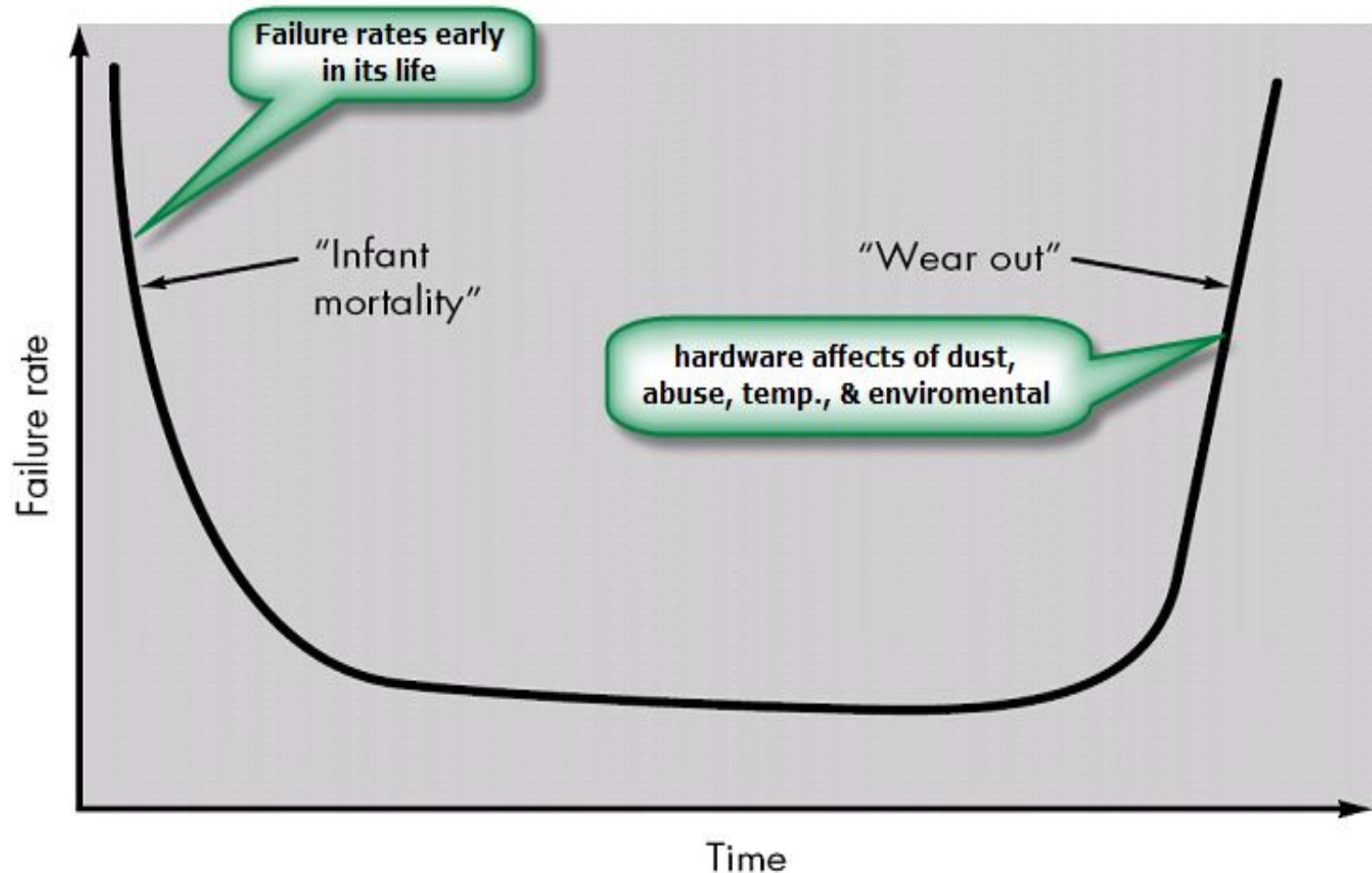
Software

- ☐ Developed/ engineered
- ☐ deteriorate
- ☐ Custom built
- ☐ Complex

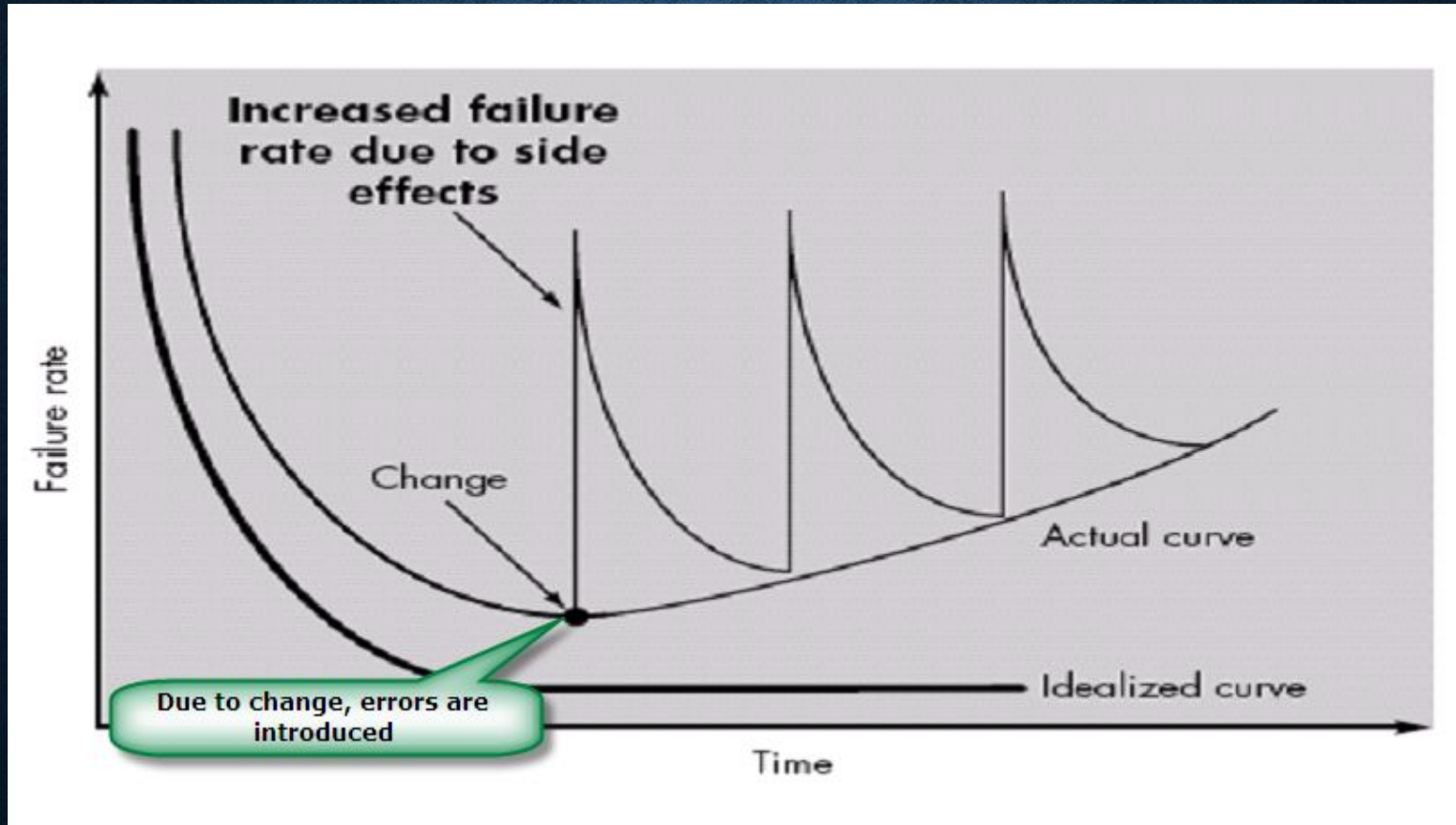
MANUFACTURING VS. DEVELOPMENT

- Once a hardware product has been manufactured, it is difficult or impossible to modify. In contrast, software products are routinely modified and upgraded.
- In hardware, hiring more people allows you to accomplish more work, but the same does not necessarily hold true in software engineering.
- Unlike hardware, software costs are concentrated in design rather than production.

FAILURE CURVE FOR HARDWARE



FAILURE CURVE FOR SOFTWARE



When a hardware component wears out, it is replaced by a spare part. There are no software spare parts. Every software failure indicates an error in design or in the process through which design was translated into machine executable code.

SOFTWARE APPLICATION DOMAINS

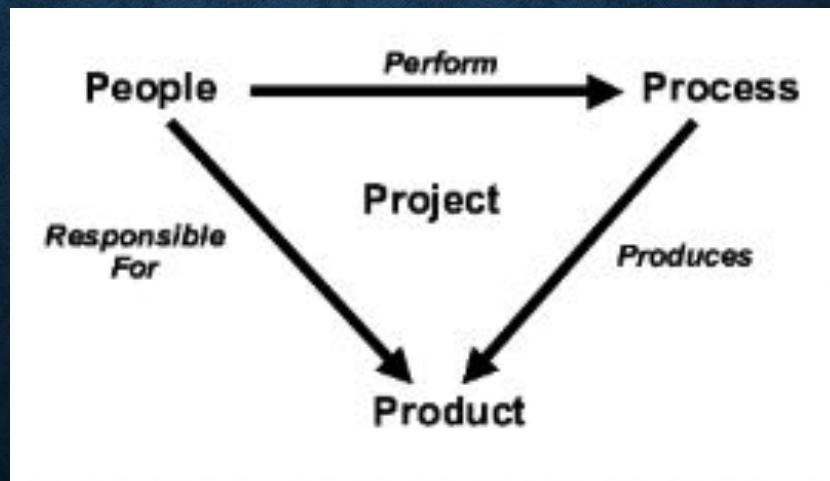
- System software
- Application software
- Engineering/scientific software
- Embedded software
- Product line software
- Web applications
- Artificial intelligence software

- **System Software:**
 - System software is a collection of programs written to service other programs.
 - Ex. Compilers, operating system, drivers etc.
- **Application Software :**
 - Application software consists of standalone programs that solve a specific business need.
 - Application software is used to control the business function in real-time.
 - Example : stock market
- **Engineering /Scientific software:**
 - Scientific software is made by complex algorithms (mostly simulation based).
 - Applications range from astronomy, stress analysis, and automated manufacturing.
 - Ex. Computer Aided Design (CAD), system stimulation etc.

- **Embedded Software:**
 - It resides in read-only memory and is used to control products and systems
 - Embedded software can perform limited functionality.
 - Ex. keypad control for a microwave oven.
- **Product line software:**
 - Designed to provide a specific capability for use by many different customers (daily routine use)- Daily billing system
 - Ex. Word processing, spreadsheet, CG, multimedia, etc.
- **Web Applications:**
 - Web apps is used to build browser-based applications. It is used to create web pages namely static and dynamic web page. It also used to link files as a hyperlink .
 - Ex. Railway Reservation System
- **Artificial Intelligence software**
 - AI software makes use of prediction algorithms to solve complex problems that are not done by straightforward analysis
 - Ex. Robotics, expert system, game playing, pattern Recognition, Disease classification, etc.

3'P OF SOFTWARE

- **Software Process:** Process is the way which is used to produce a software.
- **Software Product :** what is delivered to the customer. It may include source code, specification document, manuals, and test documentation, etc.



SOFTWARE ENGINEERING

- **According to Fritz Bauer** :An Engineering approach to develop a software is software engineering.
- **According to IEEE** : Software Engineering is a a systematic, disciplined, and cost-effective techniques for software development.

SOFTWARE ENGINEERING: A LAYERED TECHNOLOGY (4 LAYERS)



LAYERED TECHNOLOGY

A quality Focus

- Every organization is rest on its commitment to quality.
- The bedrock that supports software engineering is a quality focus.
- Example : ISO (International organizations for standardization)

IEC : (International Electro-technical Commissions)

Process:

- The processes define the tasks to be performed and the order in which they are to be performed – **what needs to be done**

LAYERED TECHNOLOGY

Methods:

- It provide the technical aspects means **how-to build** a software.
- Methods include tasks such as requirements analysis, design, coding, testing, and maintenance (support).
- There could be more than one technique to perform a task, and different techniques could be used in different situations.

Tools:

- Provide automated or semi-automated support for the process, methods and quality control.
- The tools are combined so that information created by one tool can be used by another tool, Example : ***computer-aided software engineering (CASE) Tool***

CASE TOOLS

- Planning and Requirement Analysis Phase

- Flowchart Maker

- Design Phase

- UML (Visual Paradigm)

- Coding Phase

- Eclipse, Netbeans

- Testing Phase

- Netbeans, Junit, Selenium, Cucumber

- Quality Assurance Phase

- SoapTest, Appwatch

- Maintenance Phase

- Bugzilla for defect tracking

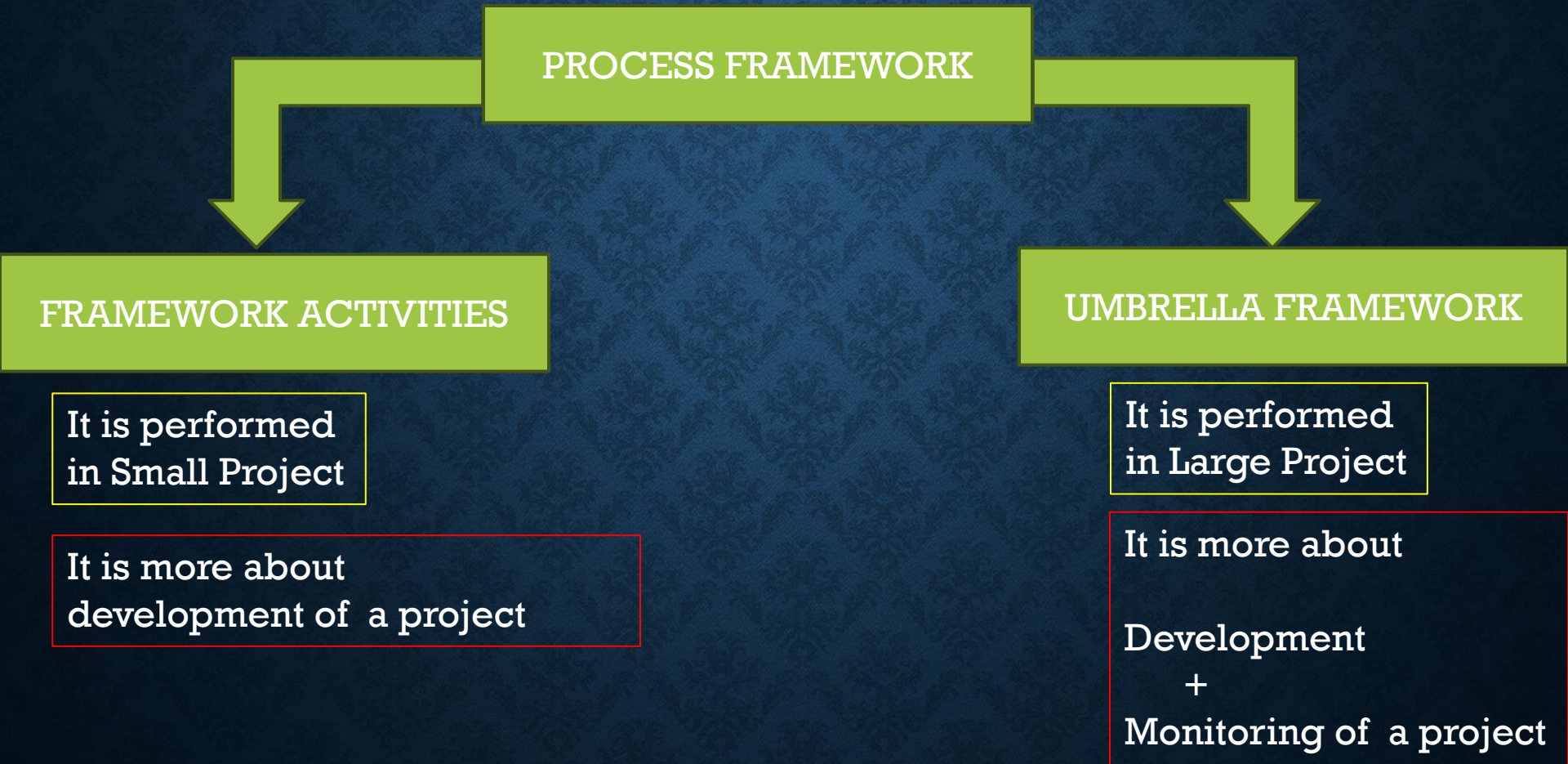
- CASE Tool is the domain of software tools used to design and implement software application
- It improves quality, cost, and productivity

PROCESS FRAMEWORK

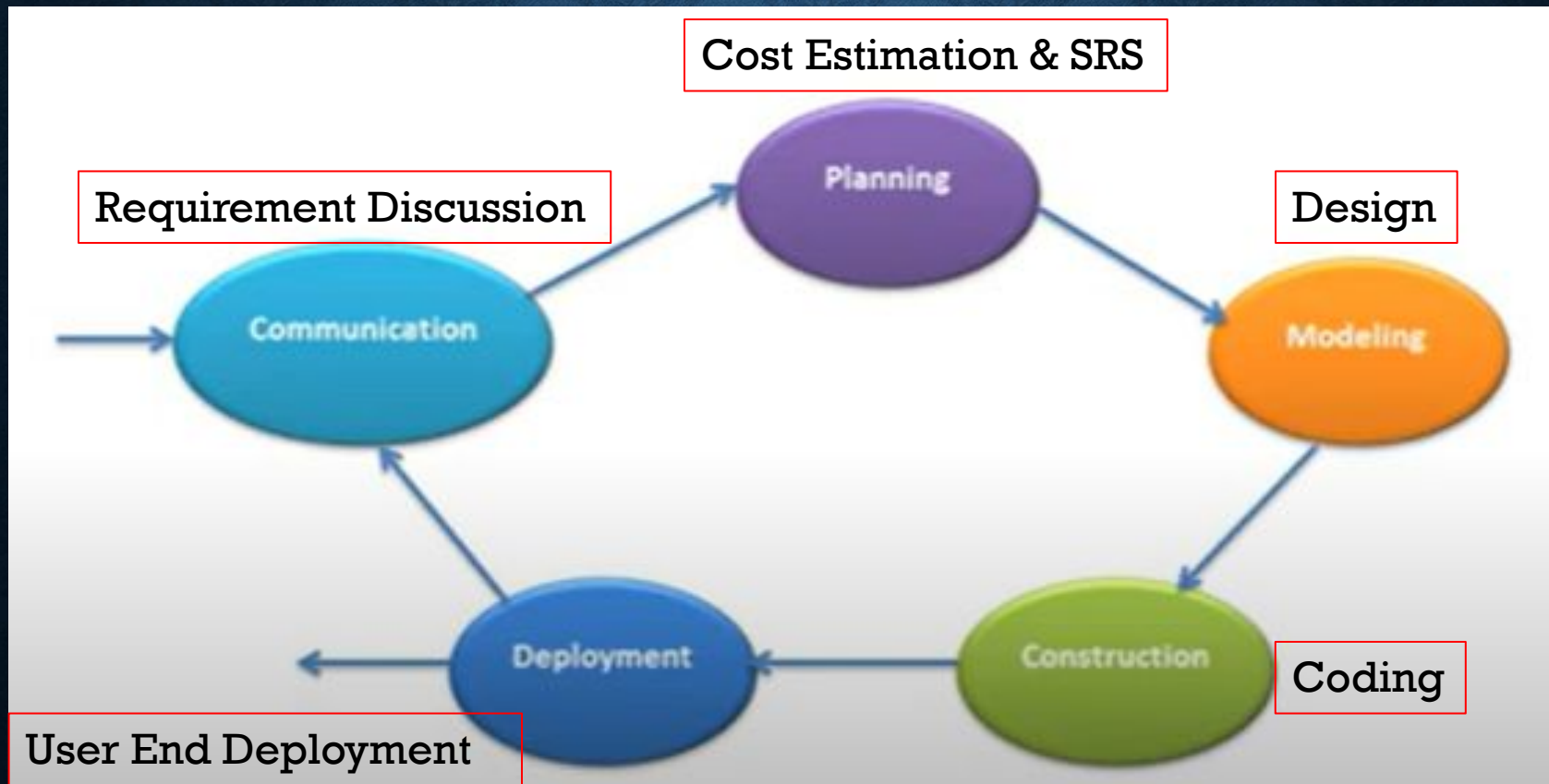
Why process framework:

- A process framework defines who is doing what, when and how to reach a certain goal.
- It consists of framework activities that are applicable to all software project development, regardless of their size or complexity.
- It also consists of an umbrella activities that are applicable across the entire software project development.

PROCESS FRAMEWORK



FRAMEWORK ACTIVITIES-CPMCD



FRAMEWORK ACTIVITIES (CPMCD)

- Communication:
 - Communication with customers, stakeholders, team members
 - It results into requirements gathering and related activities
- Planning:
 - Workflow that needs to be followed for development
 - Software project plan- Describe technical task, risk Analysis, cost estimation, work schedule.
- Modeling:
 - Developer and customer both understand the requirements (Analysis of requirements) and based on that Design of software – Top Down or Bottom Up
- Construction
 - Code generation: either manual or automated (Drag and Drop) or both
 - Testing – to uncover error in the code.
- Deployment:
 - Delivery to the customer for evaluation
 - Customer provide feedback

UMBRELLA ACTIVITIES



UMBRELLA ACTIVITIES

- Software project tracking and control
 - Assessing progress against the project plan and maintain schedule
- Formal technical reviews
 - Assessing software work products to uncover and remove errors before goes into next action or activity.
- Software quality assurance
 - Define and conducts the activities required to ensure software quality.
- Software configuration management
 - Manages the effects of change.
- Reusability management
 - Mechanisms to achieve reusable components.
- Measurement
 - Define and collects process, project, and product measures that meets customer's needs- Estimation of project.
- Risk management
 - Assesses risks that may effect that outcome of project or quality of product (i.e. software)

CAPABILITY MATURITY MODEL INTEGRATION (CMMI)

- The Software Engineering Institute (SEI) at Carnegie Mellon University has developed process meta-model to measure organization different level of process **capability and maturity**.
- The CMMI defines each process area in terms of “**specific goals**” and the “**specific practices**” required to achieve these goals.

CMMI LEVEL

Level 0 (Incomplete) –

- Process are not perform or not achieve all the goals and objectives defined by the CMMI for Level capability.

Level 1 (Performed) – All specific goals are performed as per defined by CMMI

Level 2 (Managed) –

- All level 1 criteria have been satisfied
- In addition to Level I;
 - People doing work have access to adequate resources to get job done,
 - Stakeholders are actively involved,
 - Work tasks and products are monitored, controlled, reviewed, and evaluated for conformance to process description.

Level 3 (Defined) –

- All level 2 criteria have been achieved.
- In addition;
 - management and engineering processes documented
 - standardized and integrated into organization-wide software process

CMMI LEVEL (CONT.)

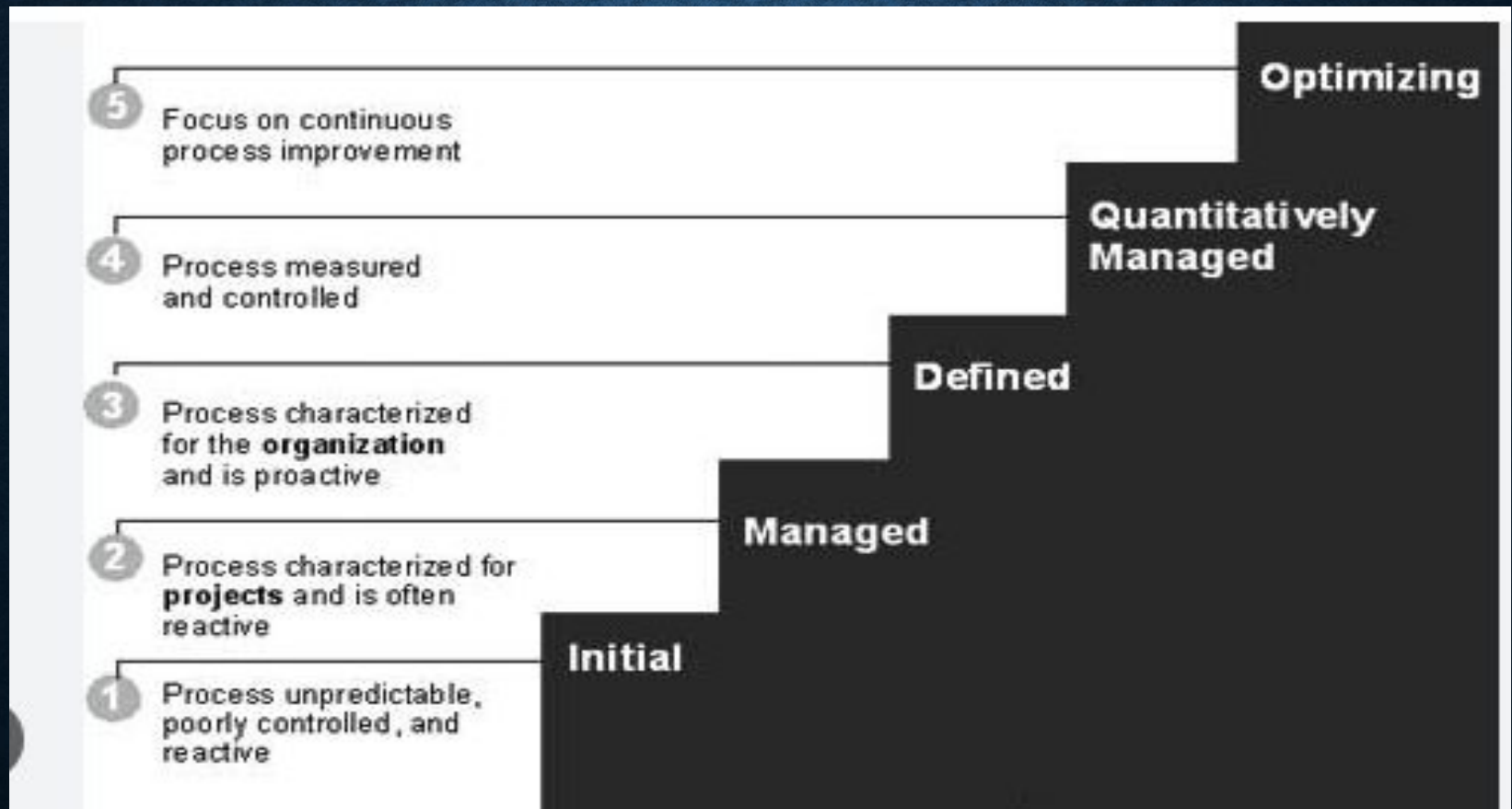
Level 4 (Quantitatively Managed) -

- All level 3 criteria have been satisfied.
- Software process and products are quantitatively understood
- Controlled using detailed measures and assessment.

Level 5 (Optimized) –

- Continuous process improvement is enabled by quantitative feedback from the process and testing innovative ideas.

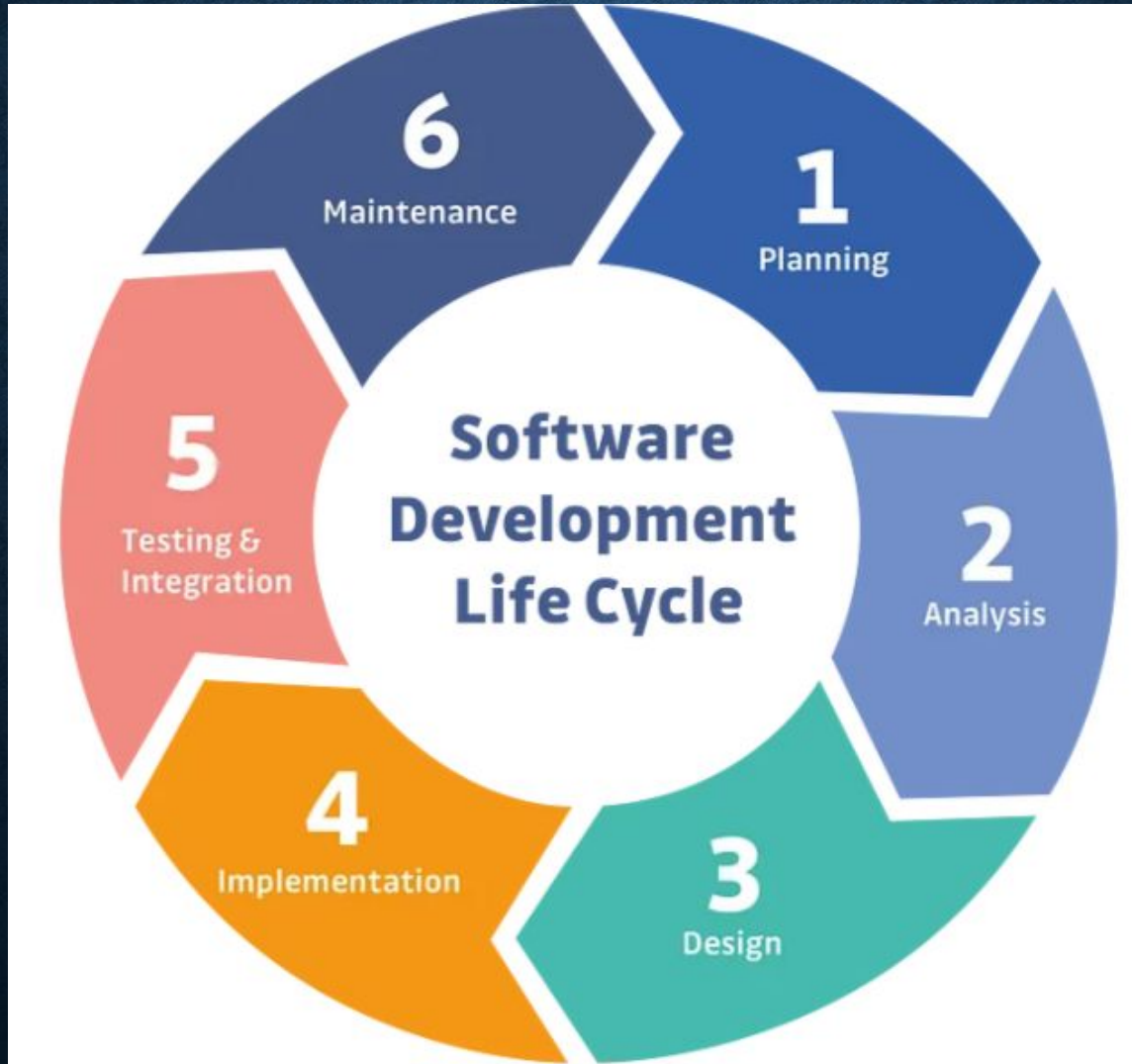
CMM LEVEL



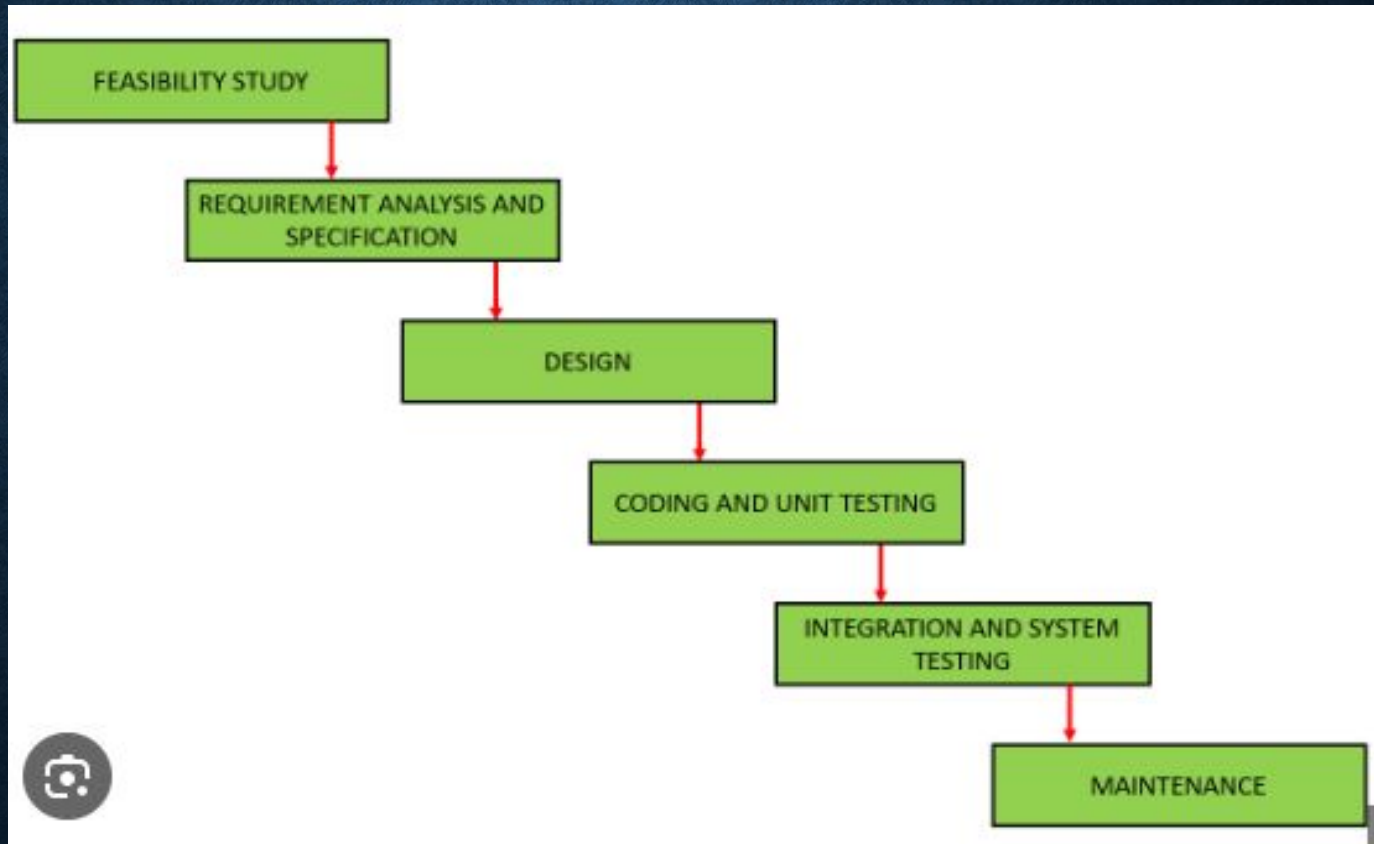
SDLC MODEL

- Waterfall Model
- Iterative Waterfall model
- V-model
- Prototype model
- Incremental Model
- Evolutionary model
- Spiral model
- Agile model

SDLC MODEL



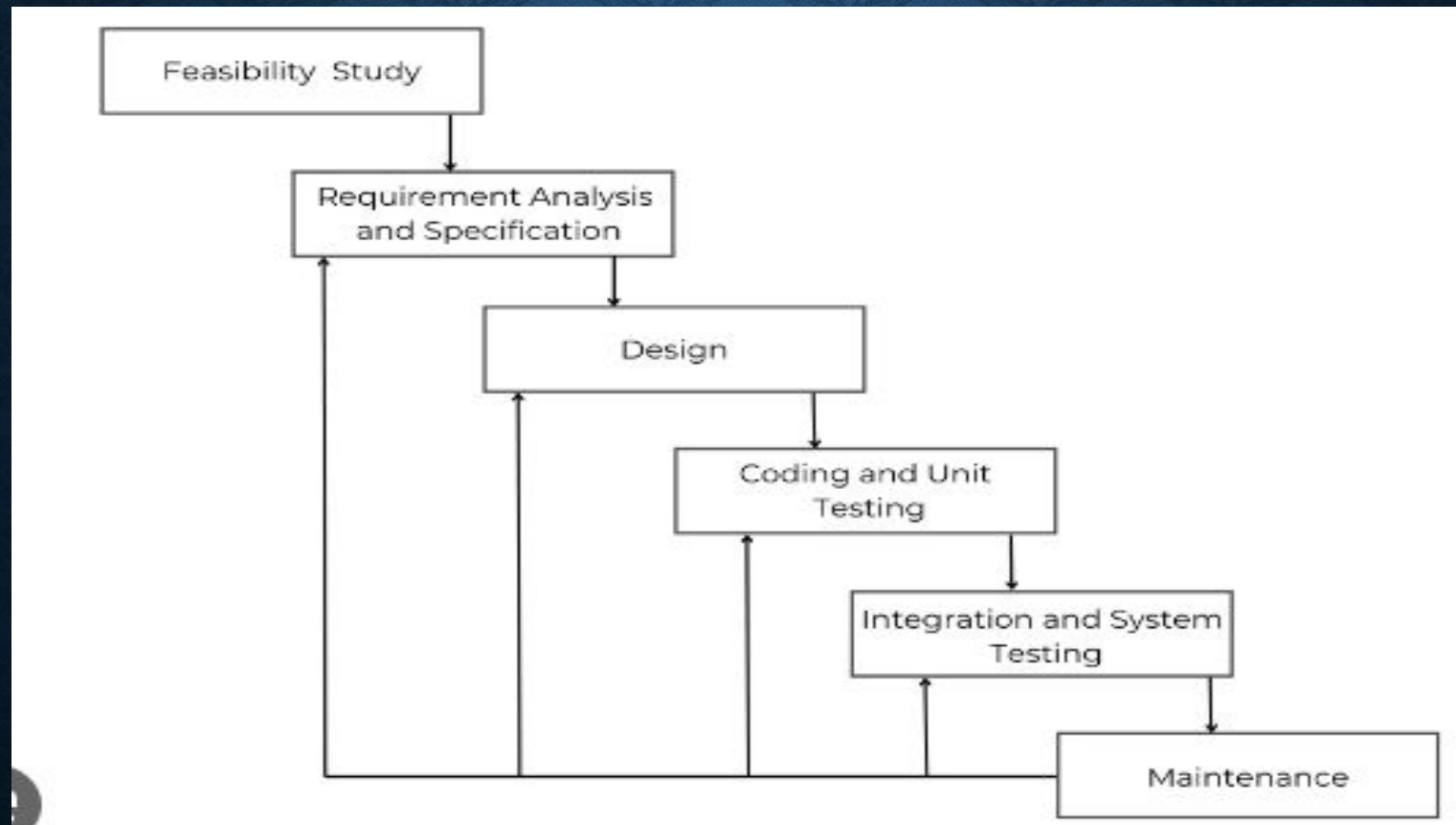
WATERFALL MODEL



CONTINUE..

- Feasibility study:- whether project is having feasibility based on cost, uses, and scalability.
- Advantage:
 - Basic model of development
 - Simple and Easy
 - Small Project
- Disadvantage
 - No Feedback
 - No Experiment
 - High Risk
 - No parallelism
 - 60% efforts goes in maintenance

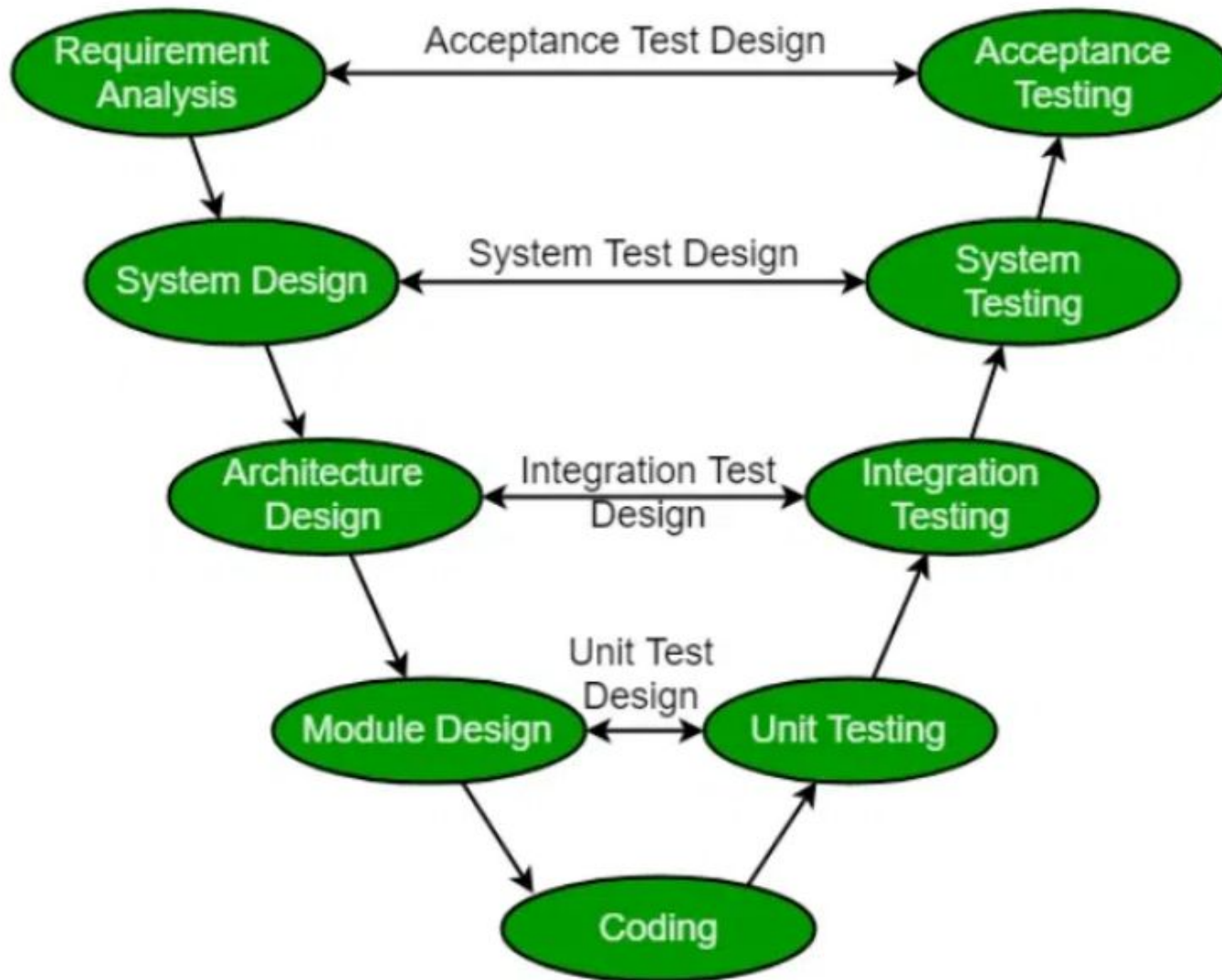
ITERATIVE WATERFALL MODEL



CONTINUE..

- Advantage
 - Basic model of development
 - Simple and Easy
 - Small Project
 - Feedback
- Disadvantage
 - No parallelism
 - No intermediary delivery of product

V-MODEL



CONTINUE..

- It is also known as verification and validation model
- Extension of waterfall model
- Testing is associated with each phase of development
- Verification phase includes (requirement analysis, system design, architecture design, module design)
- Validation phase includes (unit, integration, system, and acceptance testing)

CONTINUE..

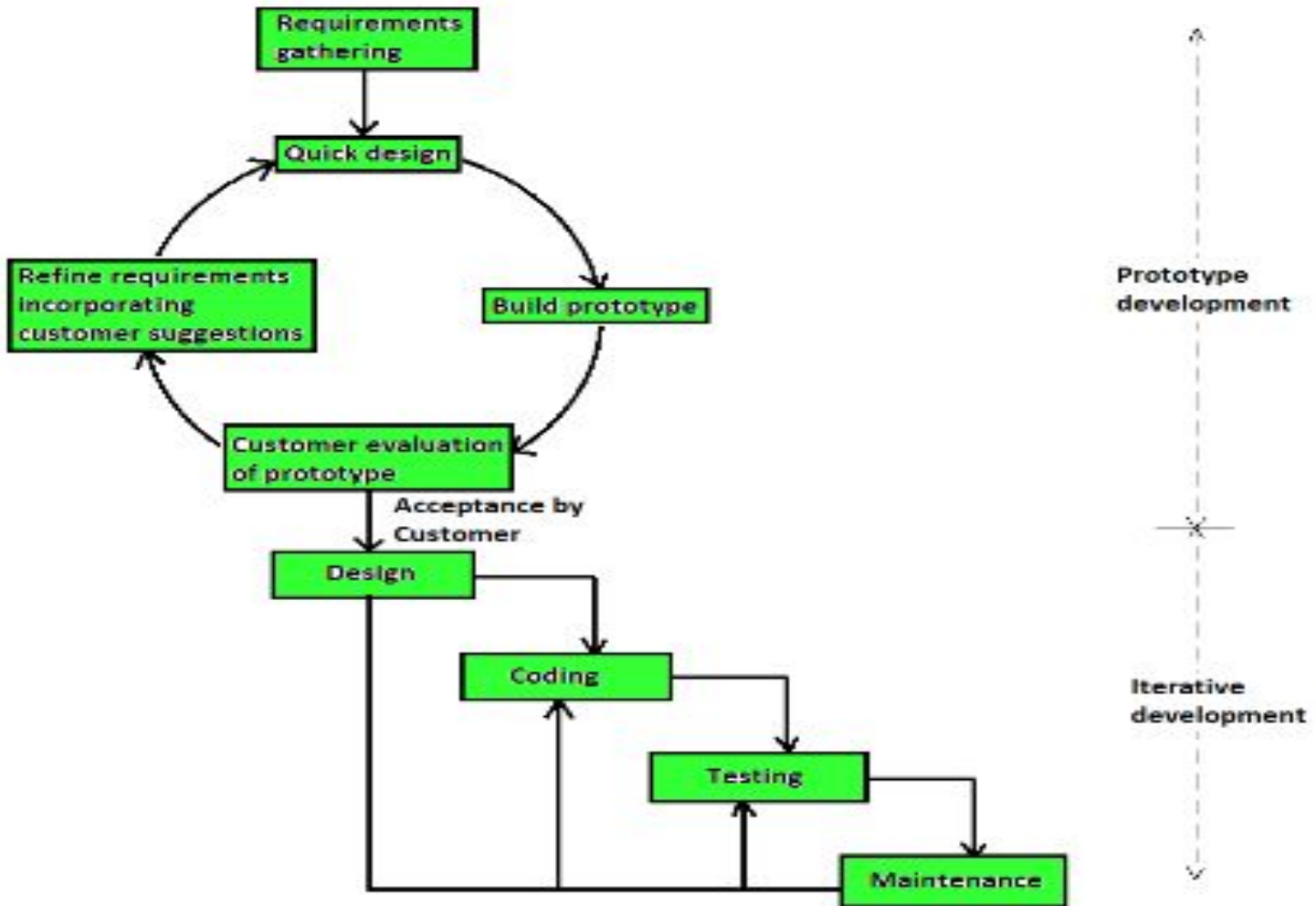
- Advantage

- Time saving
- Good understanding of project at beginning
- Every components must be testable
- Progress can be traced easily
- Proactive defect tracking

- Disadvantage

- High Risk if requirement changes dynamically
- No Early Prototypes
- Not for Complex Projects

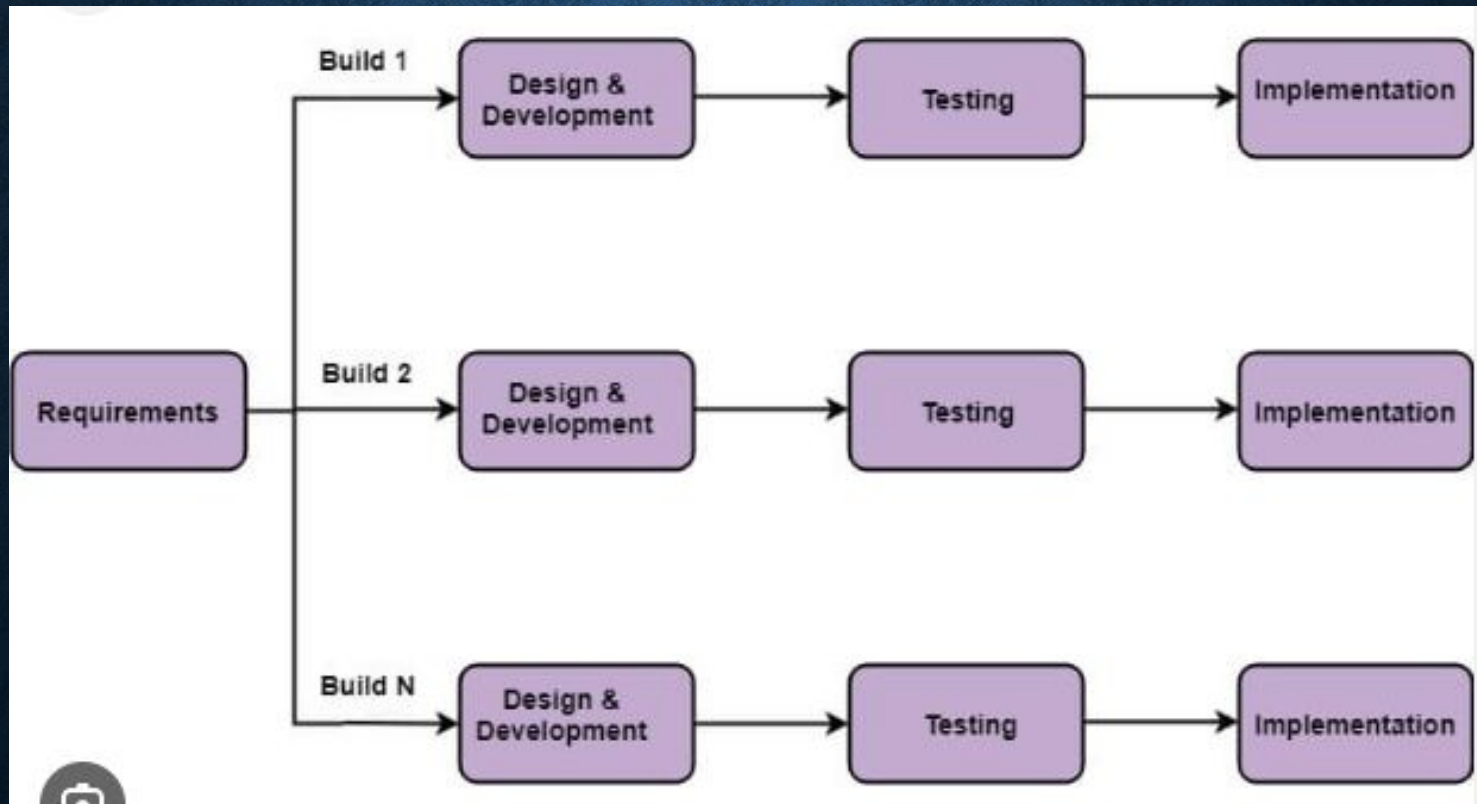
PROTOTYPE MODEL



CONTINUE..

- Advantage
 - Customer not clear with idea of working of software
 - Throw-away model
 - Good for technical and requirement risk
- Disadvantage
 - Increase in cost of development
 - Not good for complex and large projects

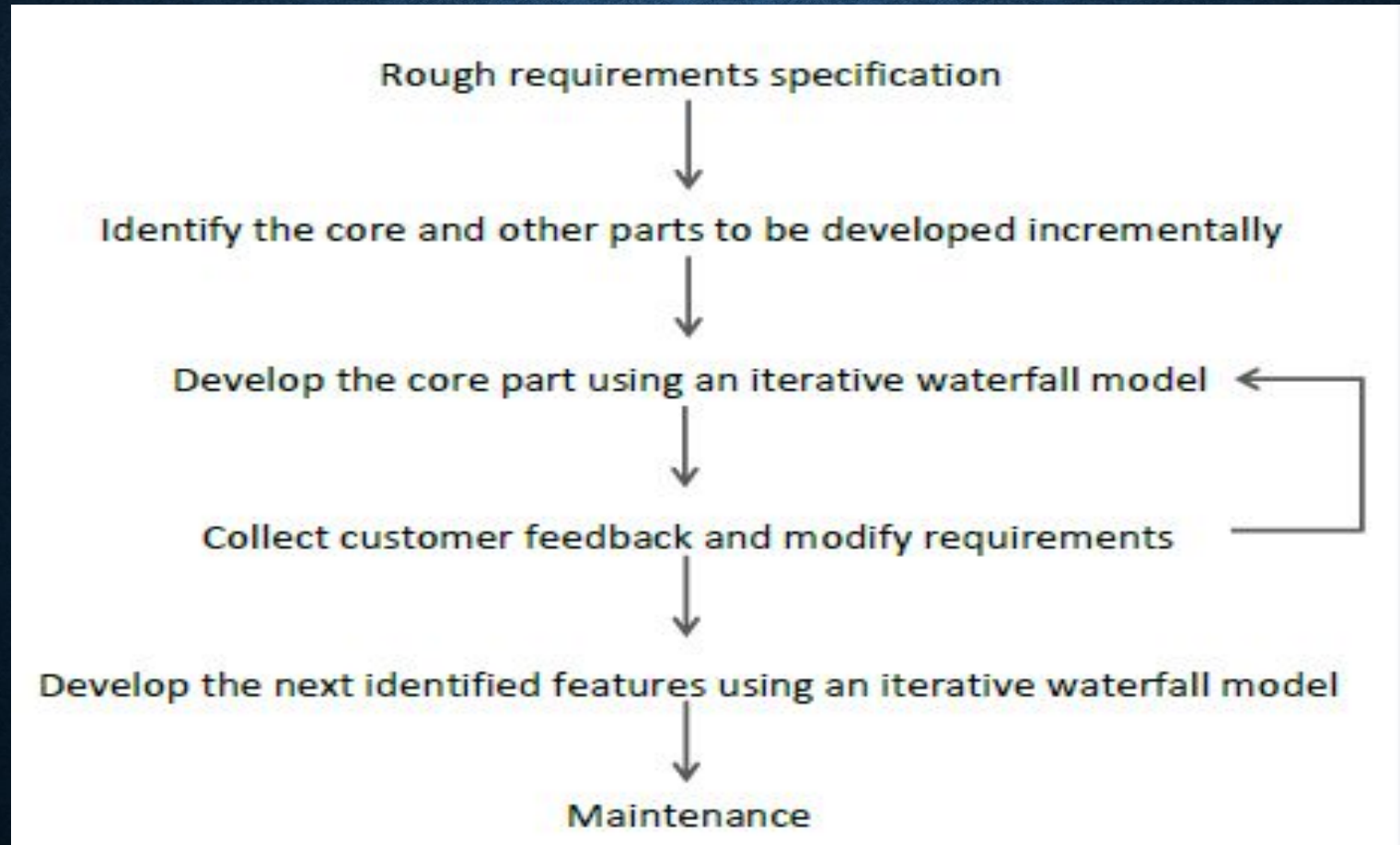
INCREMENTAL MODEL



CONTINUE..

- Advantage
 - Module by module development
 - Customer interaction maximum
 - Large project
 - Early release of product on demand
 - Flexible to change
- Disadvantage
 - Not for small project
 - Highly skilled resources are required

EVOLUTIONARY MODEL (INCREMENTAL + ITERATIVE)



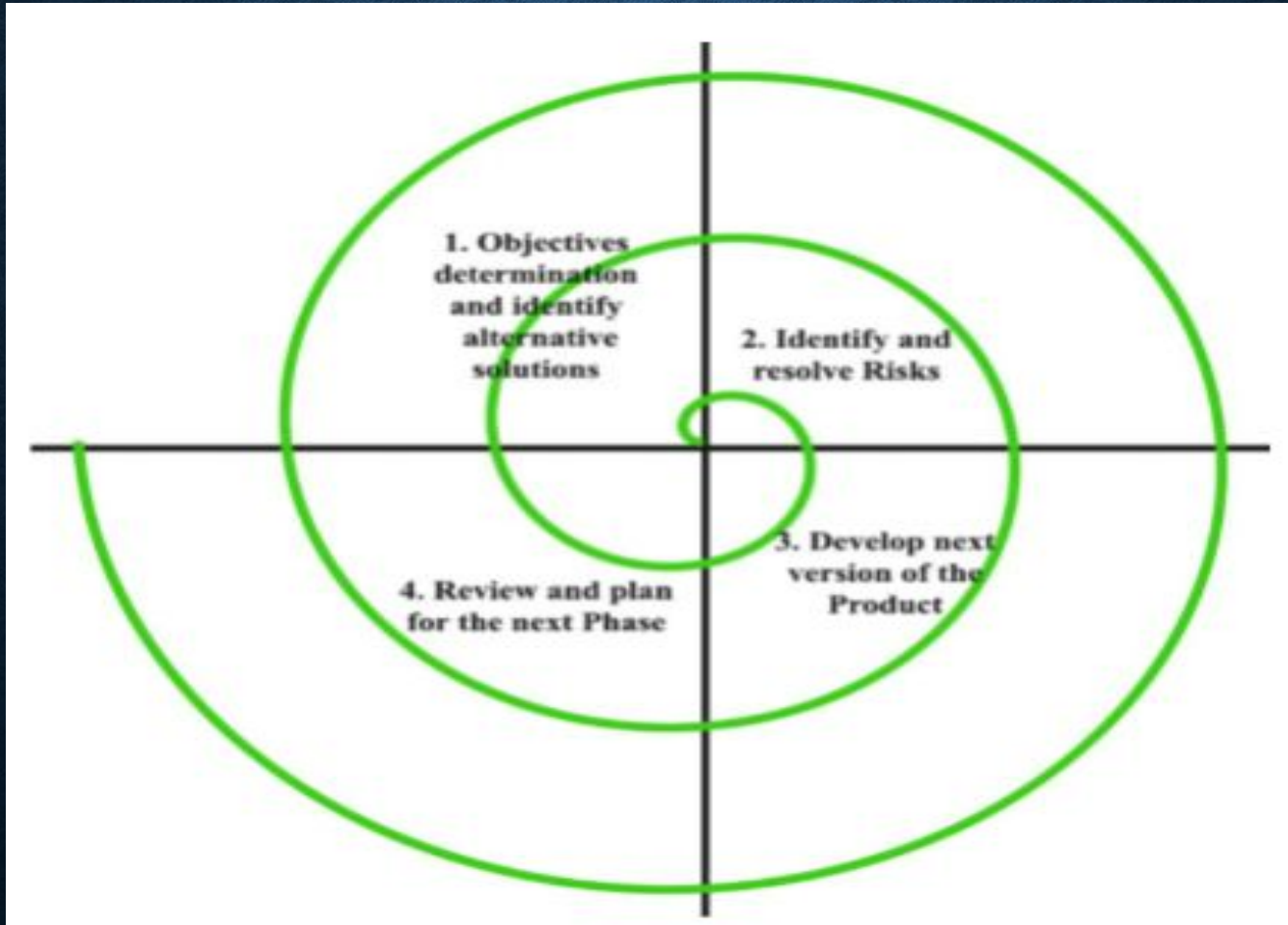
EVOLUTIONARY MODEL

- Evolutionary model is a combination of Iterative and Incremental model of software development
- Incremental model first implement a few basic features and deliver to customer. Then build a next part and deliver again and repeat this steps until the desired system is fully realized
- Iterative model is used for feedback and risk analysis
- It is also known as “design a little”, “build a little”, “test and code a little”, and “deploy a little”

EVOLUTIONARY MODEL

- Advantage
 - Customer requirements are clearly specified
 - Risk analysis is better
 - It supports changing environment
 - Initial operating time is less
 - Better suited for large projects
- Disadvantage
 - Not suitable for small project
 - Cost is high
 - Highly skilled resources are required

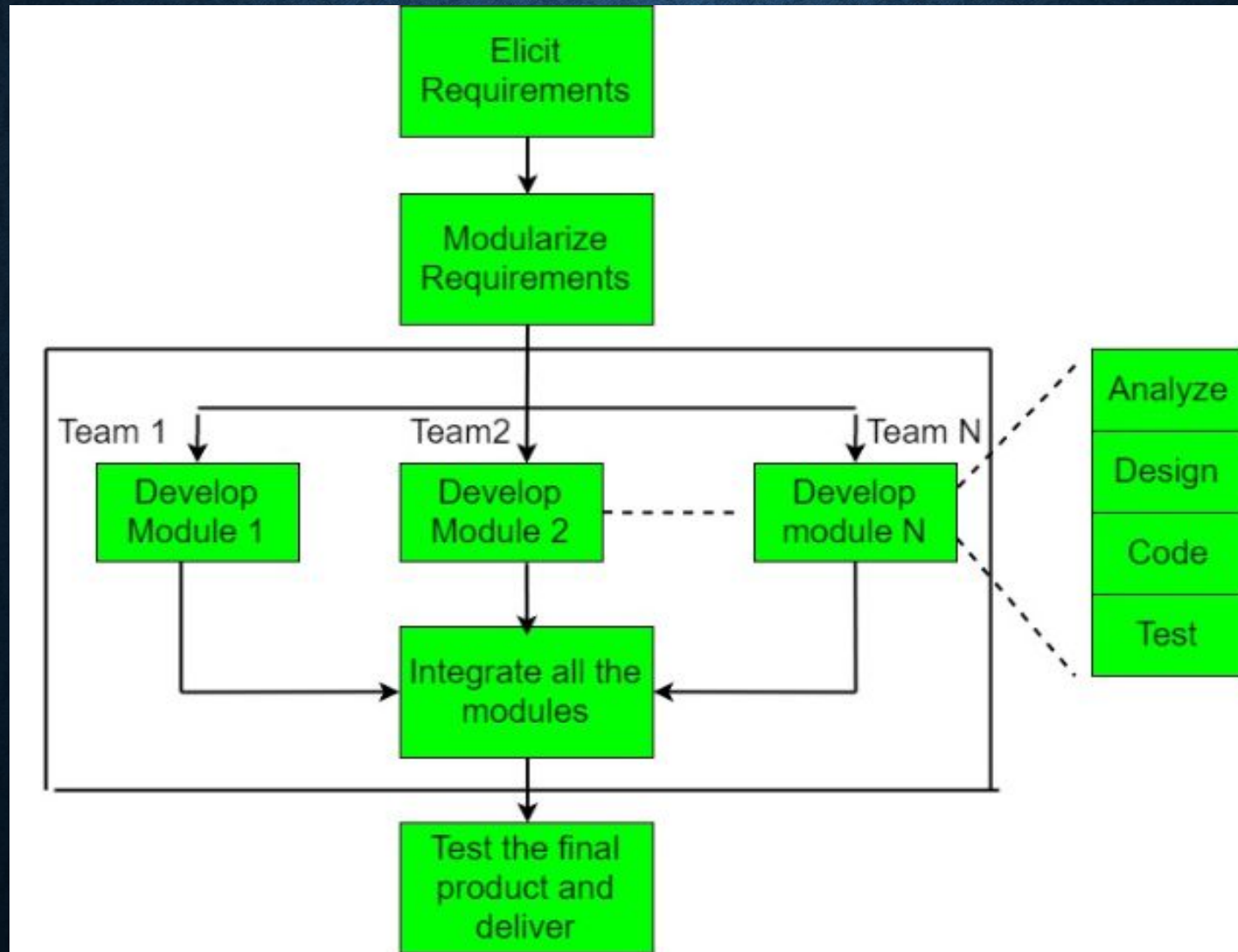
SPIRAL MODEL



CONTINUE..

- Advantage
 - Risk Handling
 - Radius of spiral= cost
 - Angular Degree = progress
 - Large project
 - Customer satisfaction
- Disadvantage
 - Complex
 - Expensive
 - Too much risk analysis

RAD MODEL



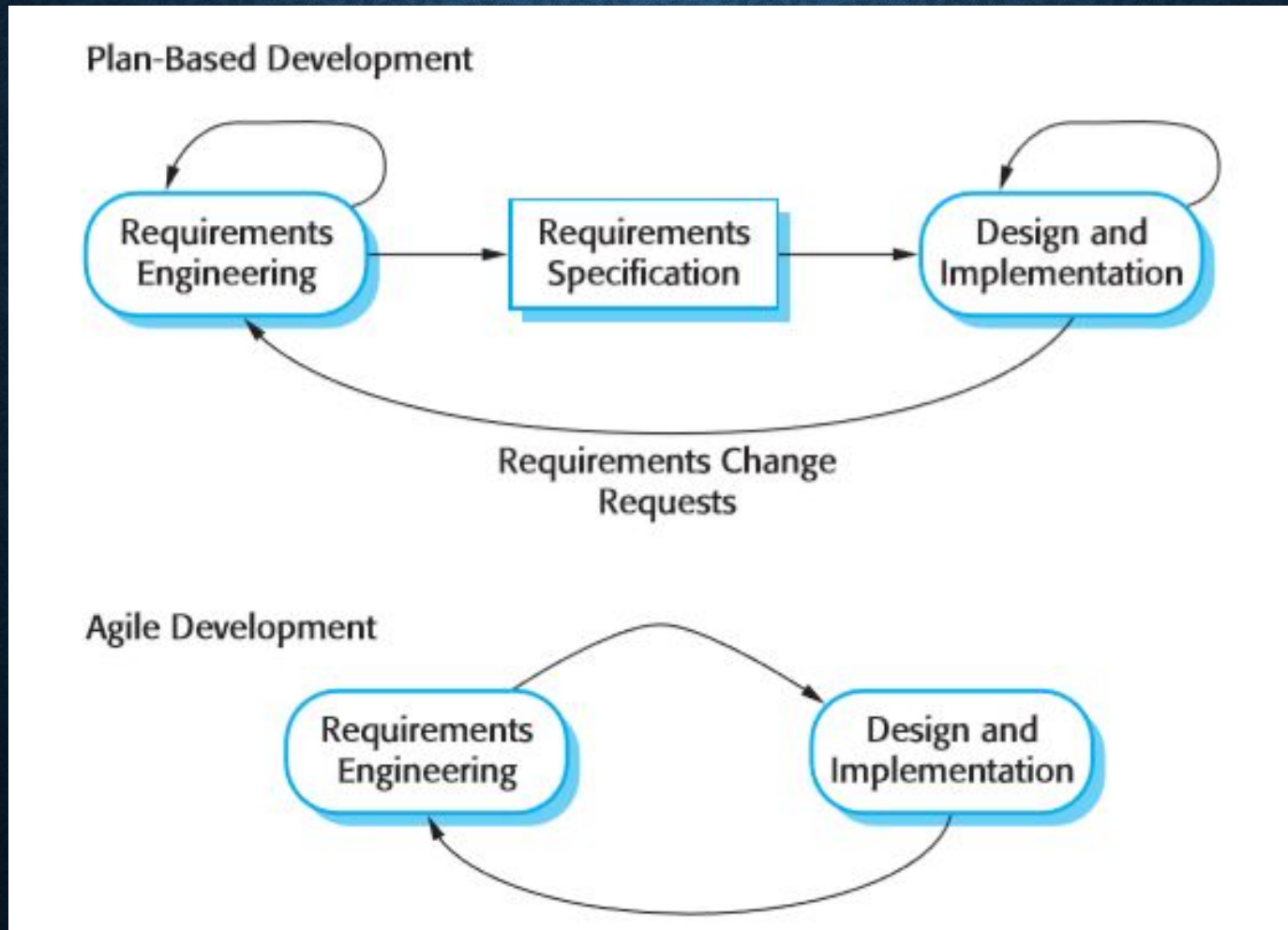
CONTINUE..

- Advantage
 - **Reduced development time:** RAD can help businesses complete more projects in a shorter amount of time (2-3 months).
 - **Customer involvement:** RAD encourages regular feedback from users and stakeholders throughout the development process.
 - **Enhanced collaboration:** RAD emphasizes communication and collaboration between developers, designers, and stakeholders.
- Disadvantage
 - **Scope creep:** User feedback can lead to continuous changes in requirements, which can cause scope creep.
 - **Project Size:** Not suitable for large projects (Managing Team is very difficult).
 - **Project Cost:** RAD may require more money spent on talented developers.

AGILE DEVELOPMENT MODELS

- Agile : **Move Quickly**
- Types of development model in Agile
 - Scrum Model
 - XP (Experimental Programming)
 - TDD (Test Driven Development)
 - FDD (Function Driven Development)
 - Crystal model

AGILE DEVELOPMENT MODEL



AGILE MANIFESTO

The Agile Manifesto

Individuals and interactions	over	Processes and Tools
Working Product	over	Comprehensive Documentation
Customer Collaboration	over	Contract Negotiation
Responding to change	over	Following a plan

That is, while there is value in the items on the right, we value the items on the left more.

www.agilemanifesto.org

What is Agility

- Effective (rapid and adaptive) response to change
- Effective communication among all stakeholders
- Drawing the customer onto the team
- Organizing a team so that it is in control of the work performed

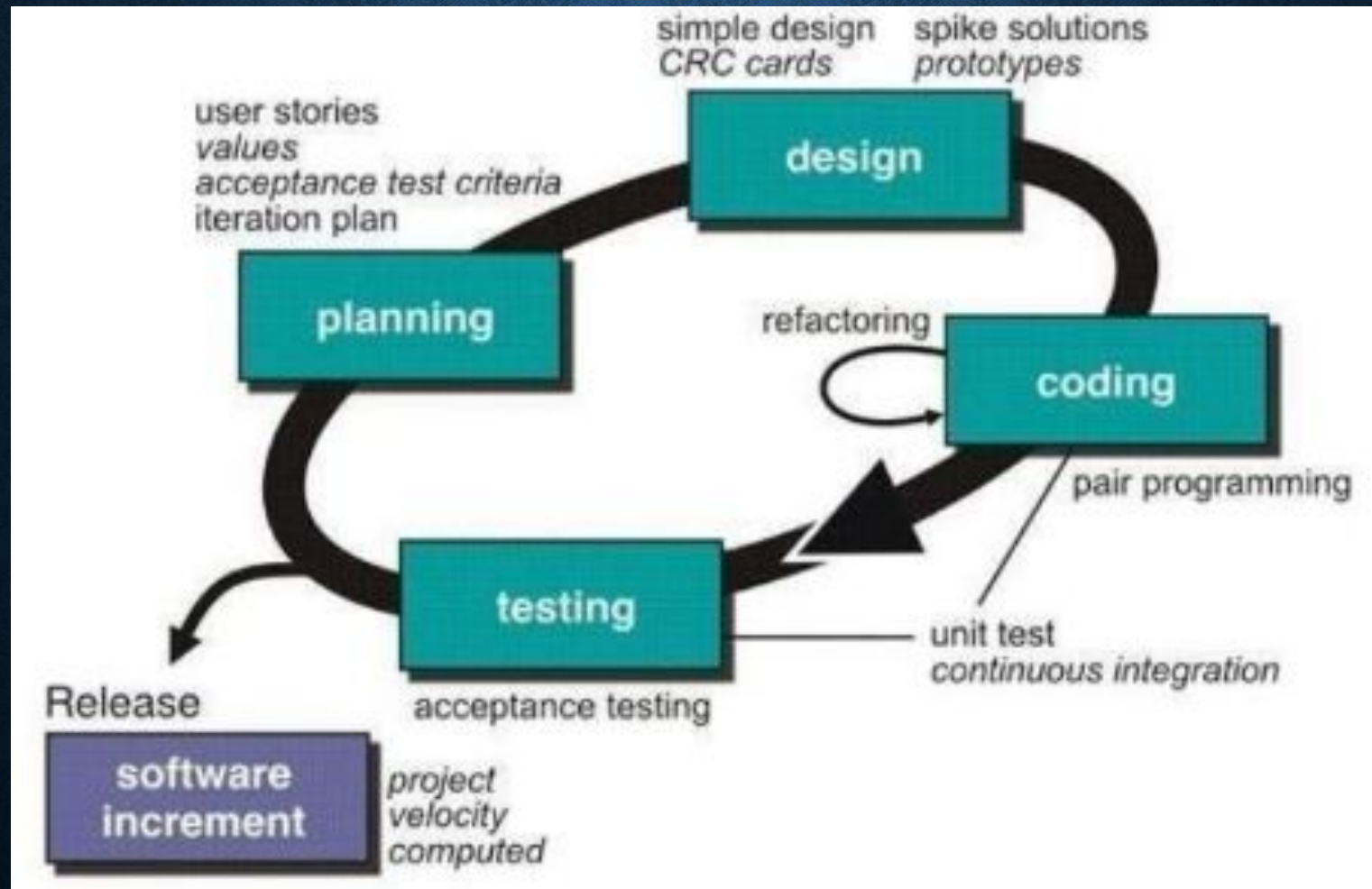
AGILE- SCRUM DEVELOPMENT

- One of the most popular agile development model is scrum model (idea came from Rugby game)
- Scrum is lightweight (document less), iterative, and incremental model
- Scrum break down the development phase into stages or cycle called “**sprint**” (term came from marathon)
- Duration of sprint is 2 week or maximum 1 months
- Development time of each sprint is dedicated thereby managing only one sprint at a time
- Scrum team has **scrum master** and **product owner** with constant communication on the daily basis
- Keywords: **Backlog, Sprint, daily Scrum, Scrum master, Product owner**
- Backlog- All the requirements are mentioned here
- Scum master- Handle the team as well as product owner

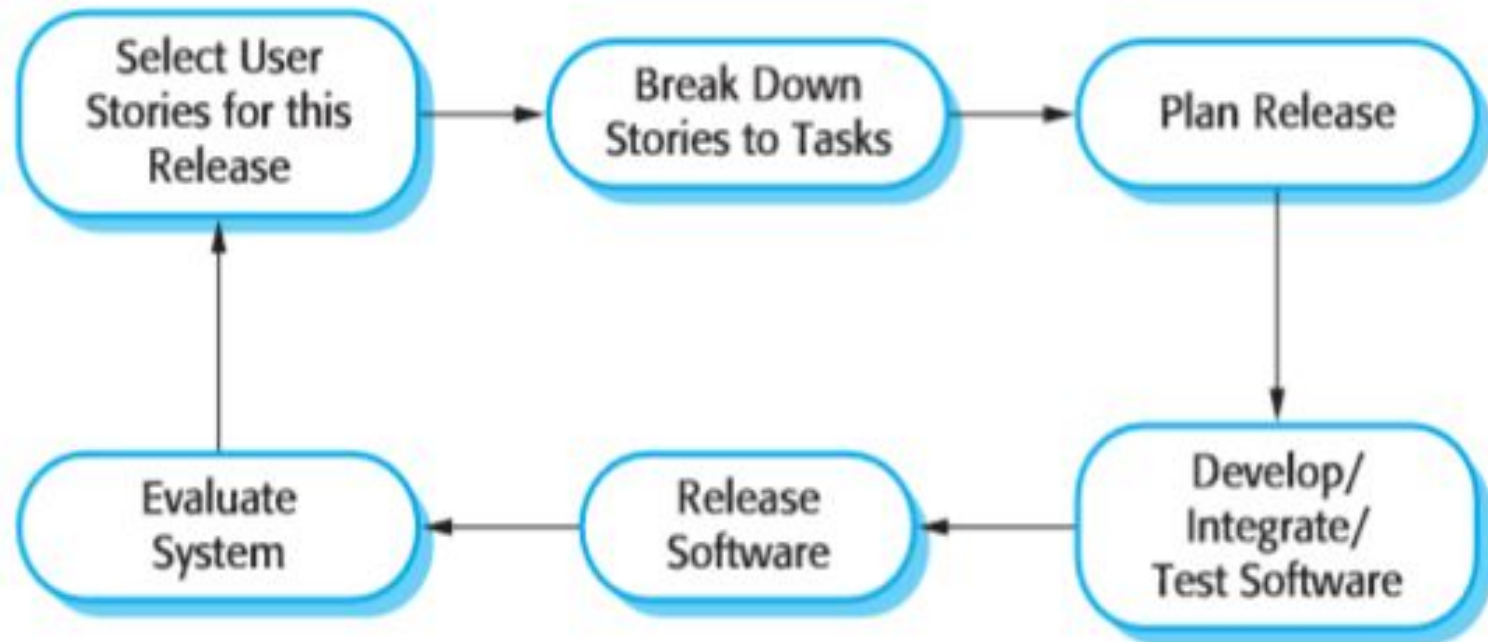
AGILE – SCRUM DEVELOPMENT

- Advantage
 - Reduce the development time
 - Scrum customer satisfaction
 - Reviewing the current sprint before moving to new one
- Disadvantage
 - More efficient for small team size
 - Maintenance problem due to less documentation

AGILE – EXPERIMENTAL PROGRAMMING



AGILE- XP



AGILE – EXPERIMENTAL PROGRAMMING

- XP Planning
 - Begins with the creation of “**user stories**”
 - Agile team assesses each story and estimates **cost**
 - Stories are grouped together and for **deliverable (working module) incrementally**
 - A **commitment** is made on delivery date
 - After the first increment “**project plan**” is used to define subsequent delivery dates for other increments (working modules)

KEY POINT IN EXPERIMENTAL PROGRAMMING

- **Story Card**- Requirement are written on story card
- **XP plan time** – 2 weeks (For development or prototype building)
- **TDD** – Write test before you write code
- **Refactoring**- Rewriting of code in better way without changing the existing code functionality
- **Pair programming**- Two developers work in pairs to check each others work and provide support
- **on-site customer** – Developer can talk any time to customer regarding his/her requirement in modules.
- **Maintenance**- : Source code documentation is discouraged as module code may change several time. (**Less maintenance**)

AGILE – EXPERIMENTAL PROGRAMMING

- Advantage

- Reduced Documentation (Based on user stories)
- Test based development (Stable development)
- Customer involvement in the iteration (for Acceptance Testing)
- Pair Programming
- Refactoring- Removal of duplication, increase cohesion, reduce coupling
- CRC Card – Class, Responsibilities, and Collaborator (To increase cohesion)

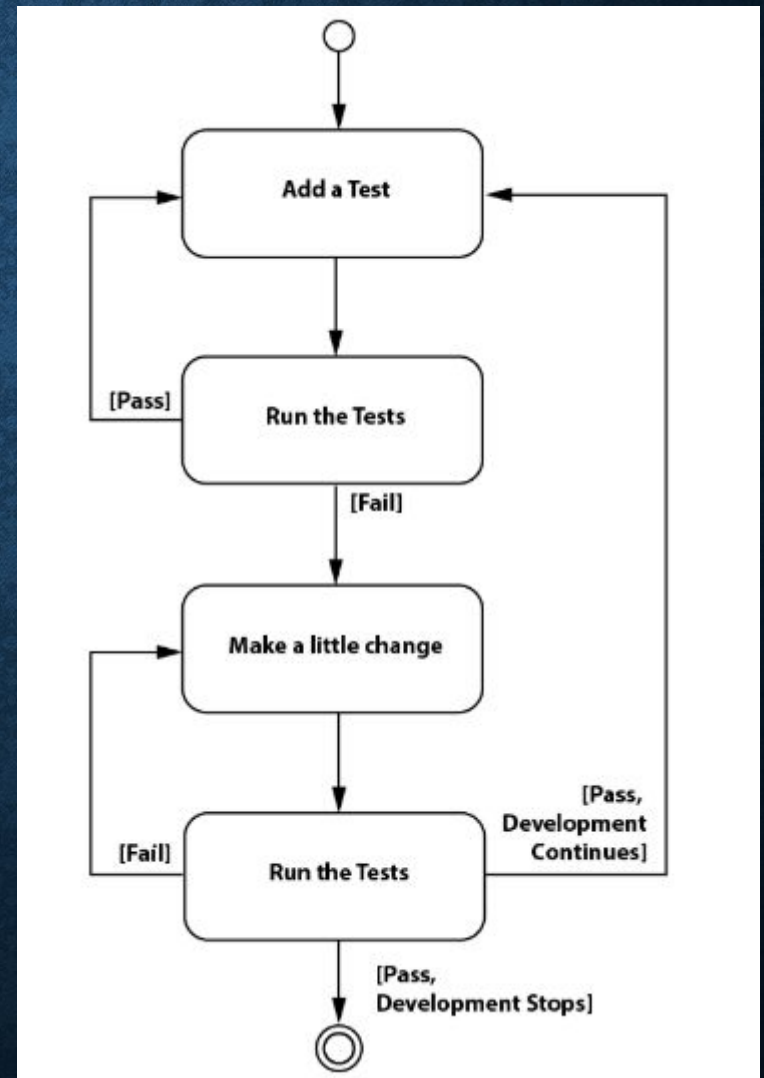
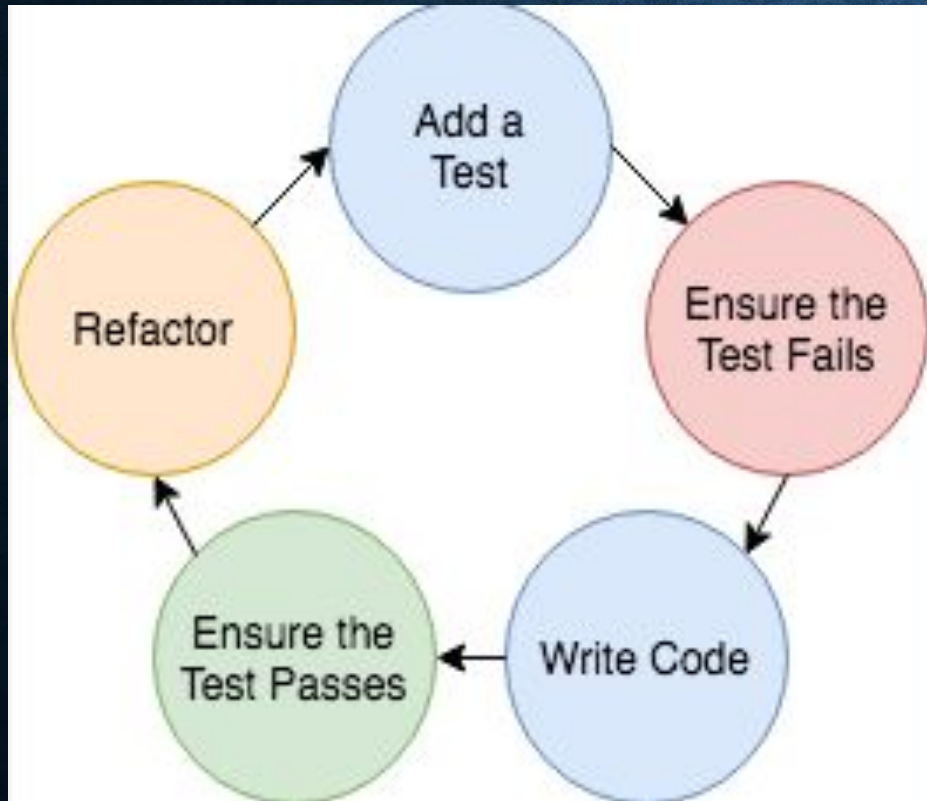
- Disadvantage

- XP is costly due to testing, refactoring, and prototype
- Require version control management (due to changes in development)

EXPERIMENTAL PROGRAMMING VS SCRUM

Keypoints	XP (Extreme Programming)	Scrum
Focus	Engineering practices & code quality	Project management & process
Primary Goal	Deliver high-quality software with frequent releases	Manage work efficiently in short iterations
Iteration Length	Very short (1–2 weeks)	Fixed sprints (2–4 weeks)
Requirement Changes	Very easily accepted, even late	Allowed but controlled via product backlog
Documentation	Minimal	Minimal but more than XP
Practices Used	Pair programming, TDD, continuous integration, refactoring	Daily Scrum, Sprint planning, review, retrospective
Roles	Programmer, Customer, Coach, Tracker	Product Owner, Scrum Master, Development Team

TEST DRIVEN DEVELOPMENT-XP



TEST DRIVEN DEVELOPMENT

- Ensures Test Fails
 - Mutation Testing
 - Value Mutation
 - Decision Mutation
 - Statement Mutation

Value Mutation

```
A=10  
B=20  
C=A ^ B  
Print C
```

Original

```
A=5  
B=20  
C=A ^ B  
Print C
```

Mutant

Decision Mutant

```
IF ( A > B)  
Print A
```

Original

```
IF ( A < B)  
Print A
```

Original

Statement Mutant

```
A=A ^ B  
B=A ^ B  
A=A ^ B
```

Original

```
B=B ^ A  
A=B ^ A  
B=B ^ A
```

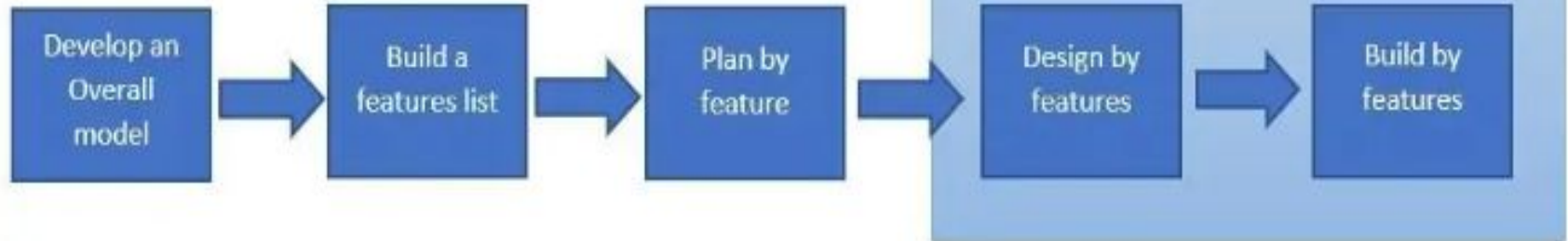
Mutant

TDD

- Advantage
 - Improved Code Quality- **reduce bugs in future**
 - Faster Feedback – **Code Correctness**
 - Better Requirement Clarity- **Applicable for TDD**
- Disadvantage
 - Time-Consuming – **Writing several test case takes time**
 - Steep Learning Curve – **Team must be familiar with Testing otherwise TDD is not suitable development process**
 - Overhead in Dynamic Requirements – **Any change in requirement makes overhead in TDD**

FEATURE DRIVEN DEVELOPMENT MODEL

FDD – 5 Stage Processes



FDD

- Advantage
 - Clear feature breakdown- suitable development if feature is planned in advance
 - User-centric approach – Feature prioritization based on customer need
 - Customer involvement in team – Requirement risk handled
- Disadvantage
 - Limited flexibility – If many changes are made by customer
 - Not well planned – unnecessary feature may be also implemented if not planned

CRYSTAL – AGILE DEVELOPMENT

Method	Team size	Project size
Crystal Clear	6 people or less	Small projects
Crystal Yellow	7–20 people	Small to medium projects
Crystal Orange	20–40 people	Medium projects
Crystal Red	40–80 people	Medium to large projects
Crystal Maroon	80–200 people	Large projects
Crystal Diamond or Crystal Sapphire	Very large teams (200+ people)	Very large projects with high criticality

CRYSTAL – AGILE DEVELOPMENT

- Advantage
 - High quality development
 - Improved customer satisfaction
 - Reduced risk
- Disadvantage
 - Lack of documentation
 - Dependence on team expertise
 - Disorganization – If team member does not have much experience

COMPARISON OF ALL SDLC MODEL/ PERSPECTIVE MODEL

Waterfall	Iterative Waterfall	Prototype	Incremental	Evolutionary	RAD	Spiral	Agile
Basic	Waterfall + Feedback	User requirement not clear	Module by module development	Incremental +	Time and cost constraints	Prototype (II-Quard)	Incremental +
Rigid		High user Involveme nt	Early deliver y of module	Iterative		Incremental (III-Quar d)	Iterative +
Inflexible		Requirement Risk is covered				+ Iterative (IV-Quar d)	Lightwei ght Develop ment
Not for large project							

QUESTIONS

- Which SDLC model is iterative and allows feedback at each iteration?

- A) Waterfall
- B) V-Model
- C) Spiral
- D) Big-Bang

(ISRO CSE – 2017)

QUESTIONS

- **What does “unit testing” primarily do?**
 - A) Tests the whole system
 - B) Tests individual modules
 - C) Tests integration
 - D) Tests performance

(ISRO CSE 2017)

QUESTIONS

- **Which phase does *not* belong to the Software Development Life Cycle?**
 - A) Testing
 - B) Maintenance
 - C) Marketing
 - D) Coding

(IBPS IT OFFICER Exam 2018)

QUESTIONS

- **Choose a disadvantage of the Waterfall model in Software Engineering:**
 - A) Not suitable when requirements change
 - B) Has undefined stages
 - C) Phases are not sequential
 - D) No deliverables

(IBPS IT OFFICER Exam 2018)

QUESTIONS

- Which SDLC model is most suitable when requirements are unclear and risk analysis is important?
 - A) Waterfall
 - B) V-Model
 - C) Spiral
 - D) Incremental

(ISRO CSE 2015,2016,2017,2018,2020)

QUESTIONS

- Which of the following is an Agile methodology?
 - A) Waterfall
 - B) Scrum
 - C) V-Model
 - D) Spiral

(Assistant Scientific Officer State PSC 2023)

QUESTIONS

- The selection of spiral model based on characteristics of requirements:

- (a) Are requirement easily understandable and defined ?
- (b) Do we change requirement quite often ?
- (c) Can we define requirement early in the cycle ?
- (d) Requirements are indicating complex to be build

Choose the correct answer

- (1) C only
- (2) B only
- (3) B and D only
- (4) A and C Only

UGC-NET 2023

Ans- 3

QUESTIONS

- The prototype model has the sequence

- (a) Customer Evaluation
- (b) Quick Design
- (c) Requirement
- (d) Implementation
- (e) Design

Choose the correct answer

- (1) C, A, D, B, E
- (2) B, C, A, D, E
- (3) C, B, D, A, E
- (4) E, B, C, D, A

UGC-NET 2023

Ans - 3

QUESTIONS

- Which of the following is not one of the principles of agile software development model
- (a) Following the plan
 - (b) Embrace change
 - (c) Customer Involvement
 - (d) Increment Delivery

UGC-NET 2018

Ans- (a)

SELECTION OF SDLC MODEL

Selection of a model is based on:

- a) Requirements
- b) Development team
- c) Users
- d) Project type and associated risk

BASED ON CHARACTERISTICS OF REQUIREMENTS

Requirements	Waterfall	Prototype	Iterative enhancement	Evolutionary development	Spiral	RAD
Are requirements easily understandable and defined?	Yes	No	No	No	No	Yes
Do we change requirements quite often?	No	Yes	No	No	Yes	No
Can we define requirements early in the cycle?	Yes	No	Yes	Yes	No	Yes
Requirements are indicating a complex system to be built	No	Yes	Yes	Yes	Yes	No

BASED ON DEVELOPMENT TEAM

Development team	Waterfall	Prototype	Iterative enhancement	Evolutionary development	Spiral	RAD
Less experience on similar projects?	No	Yes	No	No	Yes	No
Less domain knowledge (new to the technology)	Yes	No	Yes	Yes	Yes	No
Less experience on tools to be used	Yes	No	No	No	Yes	No
Availability of training if required	No	No	Yes	Yes	No	Yes

BASED ON USER PARTICIPATION

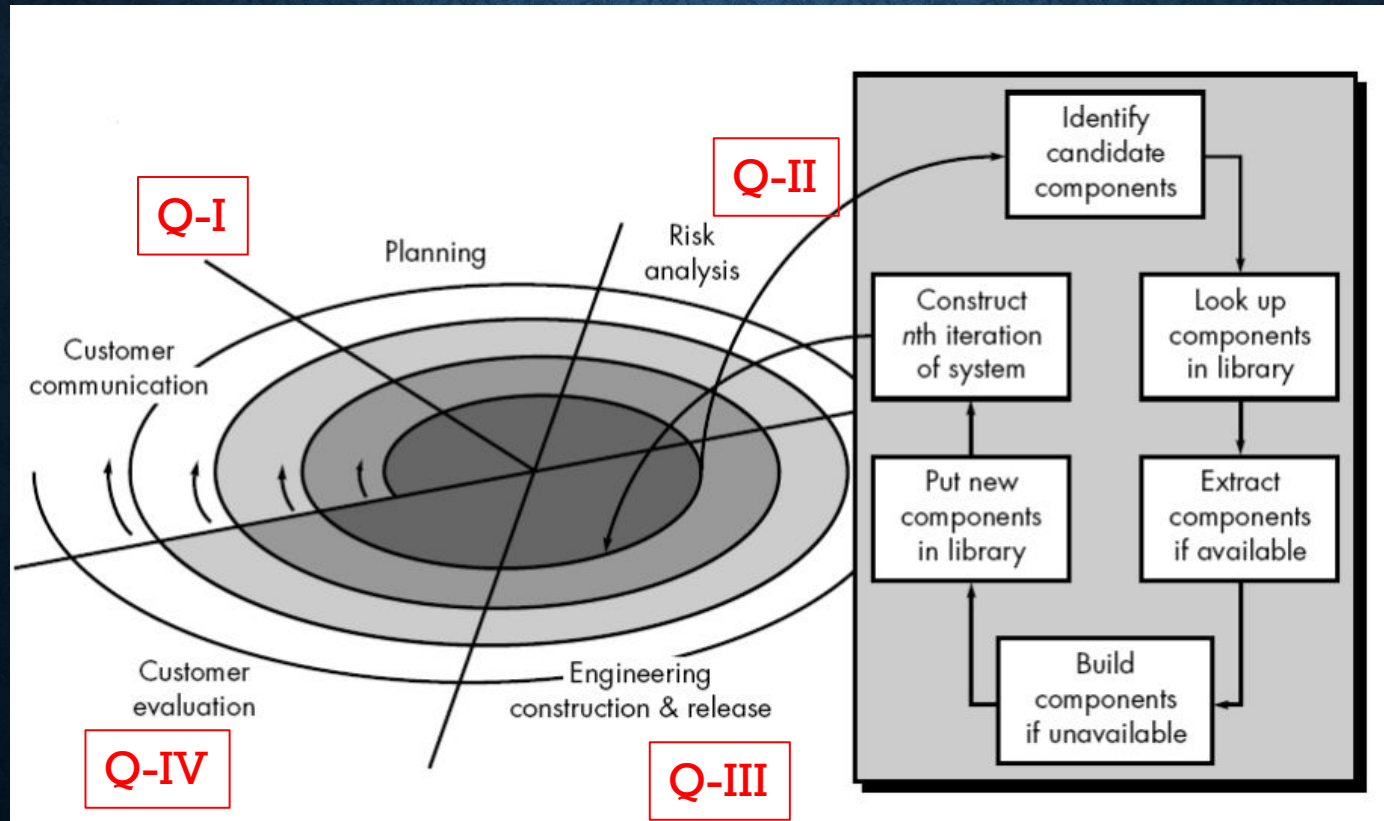
Involvement of Users	Waterfall	Prototype	Iterative enhancement	Evolutionary development	Spiral	RAD
User involvement in all phases	No	Yes	No	No	No	Yes
Limited user participation	Yes	No	Yes	Yes	Yes	No
User have no previous experience of participation in similar projects	No	Yes	Yes	Yes	Yes	No
Users are experts of problem domain	No	Yes	Yes	Yes	No	Yes

SPECIALIZED PROCESS MODEL

- Component based model
- Formal Methods model
- Aspect oriented software development

COMPONENT BASED SOFTWARE DEVELOPMENT (CBSD)

- The process to apply when re-use is a development objective



CBSD- KEY POINTS

- This process is applied when **re-use** is a development objective i.e; re-use of existing components
- These components **gives targeted functionality with well defined interfaces** which can be included in the new development
- CBSD is inspired from Spiral model of development
- **CBSD = Spiral model + Existing component and Library re -use**
- Component development model uses predefined class with similar objective (**Example Customer class, Product class, Order Class**)
- It is evolutionary in nature of development (**Incremental + Iterative**)
- Components are integrated by software architecture (**Ex- Software Programmer, Software Analyst, etc.**)

FORMAL METHODS

- This method emphasizes on mathematical specification and proof of the software which can demonstrate software correctness.

FORMAL METHODS- KEY POINTS

- **Precise Modelling:** - Mathematical expression + Automata + Logic + Set-theory etc.
- **Validation and Verification:-** Set of Rigorous activity for verification and validations
- **Tailored Approach:-** Applicable for critical domain and software development.
- **Example of Applications where Formal methods are applicable**
 - **Safety critical System-** Air craft control system, medical devices
 - **Embedded System** – Complex interaction with hardware and software
 - **Cyber Security** – protects against software threats.

ASPECT ORIENTED SOFTWARE DEVELOPMENT - AOSD

- Aspect oriented software development consists of four parts
 - Defining of software – Communication + Plan
 - Specifying of software – RA =(Functional + Non-Functional + Estimation)
 - Designing of software – Top Down + Bottom UP
 - Constructing of software – Implementation (Coding + Testing + Deployment)
- AOSD adapts the characteristics of Spiral model + Concurrent model (Evolutionally model) of development approach
- Parallel and concurrent development => Different Build development simultaneously

SOFTWARE PROCESS CUSTOMIZATION AND IMPROVEMENT

Software Process Customization

Project Type

Team Size

Team Maturity

Organizational
Constraints

Software Process Improvement

Product Quality

Productivity

Reduce Error and
Defect

Reduce Manual Work
use CASE Tool

UNIT –I (COMPLETED)

THANK YOU

UML DIAGRAMS- SEPM LAB

